

Exploratory Data Analysis on Vehicle Fuel Efficiency (MPG) Utilizing Vectorized NumPy Operations

By: Nayla Putri Ghaisani (AI18)

Submission:

- Notebook:

<https://colab.research.google.com/drive/11fSElOiOVCIUyRPcUGrRZpaoItU6rkjo?usp=sharing>

- Github upload:

<https://github.com/nylghaisani/GDGoC/tree/b9f6b06005e8f1269875bfla1754e24fefdbd896/Artificial%20Intelligence>

Part 1: Overview

The era spanning the 1970s and 1980s was a transformative period for the global automotive industry. Driven by unprecedented oil crises and skyrocketing fuel prices, automotive engineers were forced to rethink vehicle design, shifting their focus from heavy, high-displacement muscle cars to lighter, more economical vehicles. Understanding the specific physical traits that govern a vehicle's fuel efficiency measured in Miles Per Gallon (MPG) remains a classic engineering and data science puzzle. The fate of a vehicle's economy hangs in the balance of its underlying mechanical metrics.

This analysis challenges us to use vehicle records including weight, horsepower, displacement, and origin metadata to dissect the anatomy of fuel efficiency. By exploring patterns in the data using pure vectorized mathematics and linear algebra, we aim to uncover the hidden factors that drive automotive performance.

- Understanding the role of mechanical characteristics (weight, horsepower, displacement) in predicting overall fuel efficiency.
- Exploring how different manufacturing regions (USA, Europe, Japan) compare in producing consistent and economical vehicles.
- Identifying the temporal evolution of fuel efficiency across decades.

Beyond just crunching numbers, this project immerses us in the rhythm of data science where analytical coding, Exploratory Data Analysis (EDA), and mathematical modeling become the key to unraveling automotive engineering data.

Part 2: Methodology & Exploration Journey

Throughout this project, I adopted the "Vibe Coding" approach. Instead of treating the AI as an autopilot that writes entire scripts from scratch, I utilized it strictly as a technical co-pilot. I acted as the architect of the analysis structuring the logic and pipeline myself and consulted the AI to bridge syntax gaps, explain mathematical implementations, and optimize specific Pandas/NumPy operations.

As emphasized in the standard Data Science workflow, the analyst is responsible for every line of code submitted. Therefore, I rigorously evaluated, modified, and assembled the AI's snippet suggestions into the final working codebase. Below is a breakdown of this collaborative workflow from Stage 1 to Stage 4.

Stage 1: Setup and Initial Inspection

- Objective: Load the MPG dataset and ensure data integrity by evaluating missing values and calculating custom summary statistics using pure NumPy.
- AI Prompt Engineered: "I am inspecting the MPG dataset using Pandas. What is the most statistically sound method to handle a very small percentage (1.5%) of missing values in the 'horsepower' column without introducing artificial bias to the engine variance?"
- Implementation & Reflection: Instead of blindly imputing the missing data with a mean or median, the AI and I concluded that dropping the 6 empty rows (`dropna()`) was the optimal strategy to maintain strict data integrity. I then independently wrote the 100% NumPy-based loop to calculate the Mean, Median, Standard Deviation, IQR, and Outlier counts, verifying the math without relying on Pandas' `describe()` function.

Stage 2: Statistical Analysis (Strictly Vectorized)

- Objective: Perform Z-score standardization and isolate highly correlated mechanical features using pure NumPy, avoiding scikit-learn or Python loops.
- AI Prompt Engineered: "Explain the mathematical formula for Z-score standardization and how to implement it across a 2D NumPy array using broadcasting. Also, how can I use a boolean mask to prove that cars with above-average horsepower weigh more than the dataset average?"
- Implementation & Reflection: Because this stage strictly forbade high-level statistical libraries, I needed to ensure the mathematical foundation was flawless. The AI explained the broadcasting formula: $X_{\text{standardized}} = (X - \text{mean}) / \text{std}$. I implemented this vectorized formula manually using `axis=0`. For the boolean masking, I structured the conditional filtering logic to isolate the high-horsepower array, successfully proving mechanically that high-power engines significantly increase vehicle weight.

Stage 3 & 4: Visualisation & Contextual Interpretation

- Objective: Visualize relationships, distributions, and temporal trends to extract business/engineering insights.
- AI Prompt Engineered: "How do I round a 'model_year' column (which contains 2-digit years like 72, 75, 82) down to the nearest decade in Pandas so I can plot a temporal trend line chart with Seaborn?"
- Implementation & Reflection: The AI suggested utilizing the `.round(-1)` method to dynamically snap the years to their respective decades (70s, 80s). I mapped this into a groupby function to calculate the mean MPG per decade. The resulting visualization

perfectly illustrated the historical automotive shift towards fuel efficiency following the 1970s oil crisis.

Part 3: AI Integration, Advanced Math & OOP Implementation

3.1 AI-Assisted Development & Docstring Generation

3.1 Mathematical Foundations (Normal Equation & PCA)

To push the analysis beyond basic EDA, I implemented Linear Regression and Principal Component Analysis (PCA) entirely from scratch using Linear Algebra. I directed the AI to help me verify the translation of the Normal Equation into NumPy syntax ($\text{np.linalg.inv}(X.T @ X) @ X.T @ y$). By doing this manually, I proved that the manually calculated slope and intercept perfectly matched the built-in `np.polyfit()` function, validating the mathematical architecture.

3.2 Object-Oriented Refactoring (DatasetProfiler)

To transform a series of disconnected Exploratory Data Analysis scripts into a robust, reusable production pipeline, the entire EDA codebase was refactored into a single Python class named `DatasetProfiler`. This class encapsulates analytical phases into dedicated methods:

- Descriptive Statistics: Dynamic variance computation via `summary_stats()`.
- Correlation Extraction: Advanced correlation mapping via `correlation_report()`.
- Visual Dashboard: Automatically generating histograms, boxplots, and heatmaps in a single call via `plot_dashboard()`.

3.3 Production Pipeline Verification & Report Output

By wrapping the logic into a class architecture, the entire execution flow became centralized. The `generate_report()` method orchestrates the pipeline sequentially, creating an integrated data flow where each method passes validated states to aggregate the final results into a single, nested Python dictionary. When executed, it outputs a single comprehensive report payload serialized as a clean, indented JSON layout, satisfying industry standards for production-grade code.

Part 4: Reference

[1] Waskom, M., "Seaborn: statistical data visualization," Seaborn Documentation. [Online]. Available: https://seaborn.pydata.org/generated/seaborn.load_dataset.html. Accessed on: Jul. 6, 2026.

[2] Pandas Developers, "pandas.DataFrame.dropna," pandas documentation. [Online]. Available: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html>. Accessed on: Jul. 5, 2026.

[3] NumPy Developers, "Linear algebra (numpy.linalg)," NumPy Documentation. [Online]. Available: <https://numpy.org/doc/stable/reference/routines.linalg.html>. Accessed on: Jul. 6, 2026.

[4] Zahran Alvan Putra Winarko, "Module 2: Mathematical Foundation and Data Visualization for AI," Google Developer Group Bandung Institute of Technology, 2026. Accessed on: Jul. 4, 2026.

[5] NumPy Developers, "NumPy learning resources," *NumPy*. [Online]. Available: <https://numpy.org/learn/>. Accessed on: Jul. 4, 2026.

[6] Stanford University CS231n, "Python Numpy Tutorial," *CS231n: Convolutional Neural Networks for Visual Recognition*. [Online]. Available: <https://cs231n.github.io/python-numpy-tutorial/>. Accessed on: Jul. 13, 2026.

[7] GitHub, "GitHub Copilot documentation," *GitHub Docs*. [Online]. Available: <https://docs.github.com/en/copilot>. Accessed on: Jul. 13, 2026.